

Assignment #01

EECE 7205: Fundamentals of Computer Engineering

DHANUSH KARTHIK RAVICHANDRAN NUID: 003980108

Fall 2026

Problem 1: Merge Intervals

1. Understand the Problem

Goal: Given a list of intervals $[start, end]$, combine every overlapping interval and return the resulting set of non-overlapping intervals.

Input: A list of intervals, e.g. $[[1, 3], [2, 6], [8, 10], [15, 18]]$.

Output: The merged, non-overlapping intervals, e.g. $[[1, 6], [8, 10], [15, 18]]$.

Assumptions: For each interval, $start \leq end$.

Edge cases: Empty input; a single interval; intervals that all overlap into one; intervals that merely touch (e.g. $[1, 4]$ and $[4, 6]$ are treated as overlapping).

2. Design the Solution

Sort the intervals by their start value, then sweep through them once. After sorting, any interval can only overlap with the interval immediately kept before it, so a single pass is enough to merge everything. This is simpler and faster than comparing every pair.

3. Pseudocode

```
function mergeIntervals(intervals):
    if intervals is empty: return empty list
    sort intervals by start value
    merged = [ intervals[0] ]
    for i from 1 to length(intervals) - 1:
        last = last interval in merged
        if intervals[i].start <= last.end:           // overlap
            last.end = max(last.end, intervals[i].end)
        else:                                         // no overlap
            append intervals[i] to merged
    return merged
```

4. Correctness

Sorting by start guarantees that when we reach interval i , every interval that could overlap it has already been processed. We only ever compare the current interval against the last one kept in the result. If it overlaps, we extend that interval's end to the larger of the two ends (which correctly

absorbs fully-nested intervals); if it does not overlap, no later interval can overlap the kept one either, so we safely start a new one. Empty input returns an empty list, and a single interval is returned unchanged.

5. Complexity

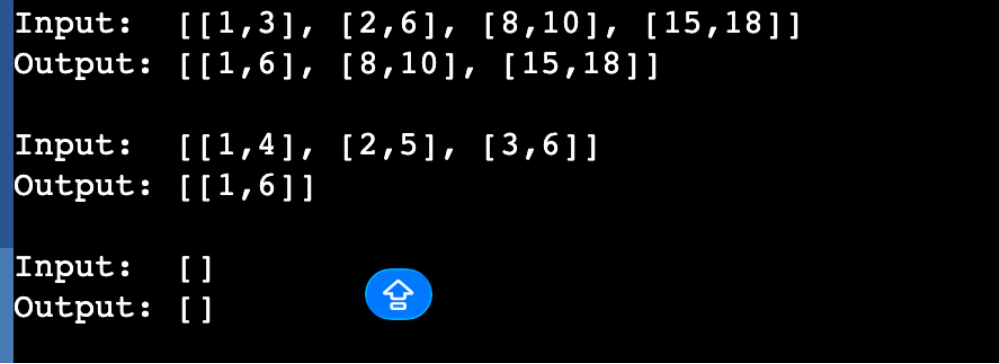
Time: $O(n \log n)$ in all cases, dominated by the sort; the single sweep is $O(n)$.

Space: $O(n)$ for the output list (or $O(\log n)$ auxiliary if the output is not counted).

6. Test Cases

- Given example: $[[1, 3], [2, 6], [8, 10], [15, 18]] \rightarrow [[1, 6], [8, 10], [15, 18]]$
- All overlap: $[[1, 4], [2, 5], [3, 6]] \rightarrow [[1, 6]]$
- Edge case (empty): $[] \rightarrow []$

7. Execution Screenshot



```
Input:  [[1,3], [2,6], [8,10], [15,18]]
Output: [[1,6], [8,10], [15,18]]

Input:  [[1,4], [2,5], [3,6]]
Output: [[1,6]]

Input:  []
Output: []
```

Problem 2: Sort Colors (0s, 1s, 2s)

1. Understand the Problem

Goal: Given an array containing only the values 0, 1, and 2, sort it in place so all 0s come first, then all 1s, then all 2s.

Input: An array such as $[2, 0, 2, 1, 1, 0]$.

Output: The sorted array $[0, 0, 1, 1, 2, 2]$.

Constraints: Only the values 0, 1, 2 appear. Two solutions are required: a counting-based approach and the one-pass Dutch National Flag algorithm (no `std::sort`).

Edge cases: Empty array; array already sorted; array with only one distinct value.

2. Design the Solution

Counting sort: Because there are only three possible values, we can count how many of each occur, then overwrite the array with that many 0s, 1s, and 2s. This takes two passes.

Dutch National Flag (DNF): Maintain three regions with pointers `low`, `mid`, and `high`. Everything before `low` is 0, everything after `high` is 2, and the middle is being processed. This sorts in a single pass with no extra array.

3. Pseudocode

```
// Version 1: Counting
function sortColorsCounting(nums):
    count = [0, 0, 0]
    for x in nums: count[x] += 1
    index = 0
    for value in 0..2:
        repeat count[value] times:
            nums[index] = value; index += 1
    return nums

// Version 2: Dutch National Flag
function sortColorsDNF(nums):
    low = 0; mid = 0; high = length(nums) - 1
    while mid <= high:
        if nums[mid] == 0: swap(nums[low], nums[mid]); low++; mid++
        else if nums[mid] == 1: mid++
        else: swap(nums[mid], nums[high]); high--
    return nums
```

4. Correctness

Counting: Every element is counted once, and the array is rewritten with exactly that many of each value in ascending order, so the result is fully sorted.

DNF: The loop invariant is that indices $< \text{low}$ hold 0s, indices $> \text{high}$ hold 2s, and $\text{low}.. \text{mid}-1$ hold 1s. A 0 is swapped down and both low and mid advance; a 1 is left in place and mid advances; a 2 is swapped up and high retreats without advancing mid (so the newly swapped-in value is re-examined). When $\text{mid} > \text{high}$ every element is placed. Empty and single-value arrays are handled naturally since the loop simply does little or no work.

5. Complexity

Counting: Time $O(n)$ (two passes); space $O(1)$ (a fixed count array of size 3).

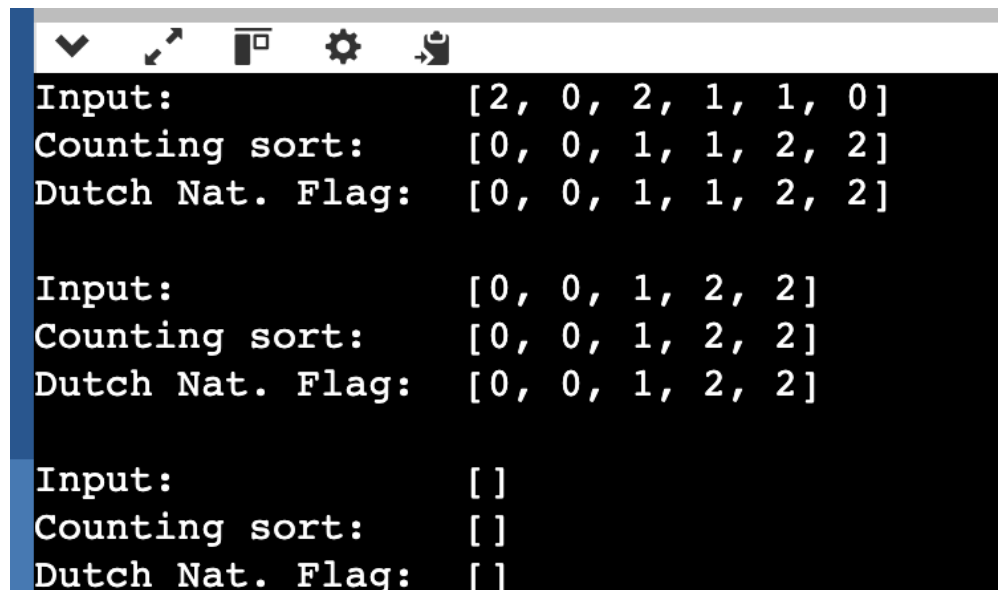
DNF: Time $O(n)$ (single pass); space $O(1)$ (in place).

Both are linear; DNF is preferred when a single in-place pass is desired.

6. Test Cases

- Given example: $[2, 0, 2, 1, 1, 0] \rightarrow [0, 0, 1, 1, 2, 2]$
- Already sorted: $[0, 0, 1, 2, 2] \rightarrow [0, 0, 1, 2, 2]$
- Edge case (empty): $[] \rightarrow []$

7. Execution Screenshot

A screenshot of a terminal window with a dark background and light-colored text. The terminal shows three test cases for a sorting algorithm. Each case displays the input array, the result of counting sort, and the result of the Dutch National Flag algorithm. The first case uses the input [2, 0, 2, 1, 1, 0], the second uses [0, 0, 1, 2, 2], and the third uses an empty array []. The results for counting sort and the Dutch National Flag algorithm are identical to the input arrays in all three cases. The terminal window has a toolbar at the top with icons for a dropdown menu, a cursor, a window, a gear, and a clipboard.

```
Input:          [2, 0, 2, 1, 1, 0]
Counting sort:  [0, 0, 1, 1, 2, 2]
Dutch Nat. Flag: [0, 0, 1, 1, 2, 2]

Input:          [0, 0, 1, 2, 2]
Counting sort:  [0, 0, 1, 2, 2]
Dutch Nat. Flag: [0, 0, 1, 2, 2]

Input:          []
Counting sort:  []
Dutch Nat. Flag: []
```

Problem 3: Largest Number

1. Understand the Problem

Goal: Given a list of non-negative integers, arrange them such that their concatenation forms the largest possible number, and return it as a string.

Input: A list of non-negative integers, e.g. [10, 2].

Output: A string representing the largest concatenated number, e.g. "210".

Assumptions: Input numbers are non-negative integers.

Edge cases: Empty input; single element; all zeros (e.g. [0, 0] should output "0", not "00").

2. Design the Solution

Convert all integers to strings and sort them using a custom comparator. For any two numbers A and B , compare their concatenated forms $A + B$ and $B + A$. If $A + B > B + A$, A must precede B in the final ordering. After sorting, concatenate all elements. If the leading element is "0", return "0".

3. Pseudocode

```
function largestNumber(nums):
    if nums is empty: return ""
    convert all numbers in nums to strings
    sort strNums using comparator(a, b):
        return (a + b) > (b + a)
    if strNums[0] == "0":
        return "0"
    return concatenation of all strings in strNums
```

4. Correctness

The comparator $A + B > B + A$ satisfies transitivity, forming a strict total ordering over the strings[cite: 5]. Because local pair-wise comparisons are consistent, sorting guarantees that every adjacent pair satisfies this optimal ordering, which directly implies a globally optimal concatenation[cite: 5]. Handling the leading "0" prevents redundant trailing zeros for all-zero inputs[cite: 5].

5. Complexity

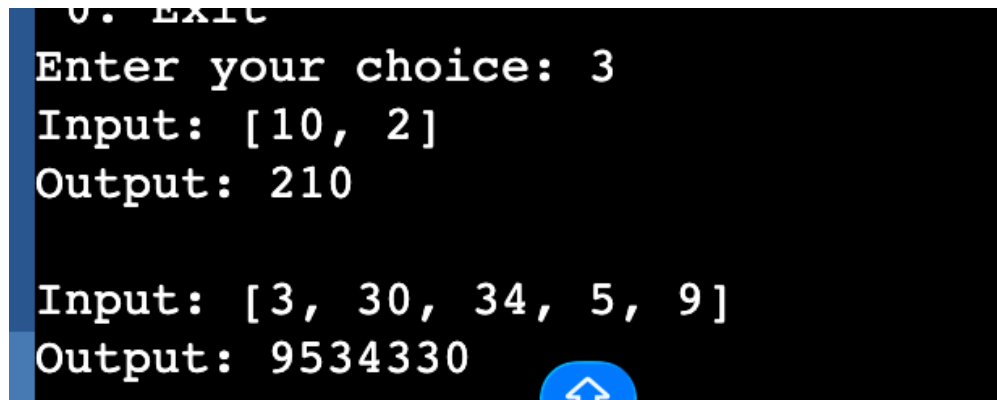
Time: $O(n \log n \cdot k)$, where n is the number of integers and k is the maximum number of digits per integer, dominated by string comparisons during sorting.

Space: $O(n \cdot k)$ to store string representations of the numbers.

6. Test Cases

- Given example: $[10, 2] \rightarrow "210"$
- Multi-digit mix: $[3, 30, 34, 5, 9] \rightarrow "9534330"$
- Edge case (all zeros): $[0, 0] \rightarrow "0"$

7. Execution Screenshot



```
0. EXEC
Enter your choice: 3
Input: [10, 2]
Output: 210

Input: [3, 30, 34, 5, 9]
Output: 9534330
```

Problem 4: Group Anagrams

1. Understand the Problem

Goal: Given an array of strings, group the anagrams together. The order of output groups does not matter, but words within each group must retain their original relative order.

Input: A vector of strings, e.g. `["eat", "tea", "tan", "ate", "nat", "bat"]`.

Output: Grouped anagrams, e.g. `[["bat"], ["tan", "nat"], ["eat", "tea", "ate"]]` (group order may vary).

Assumptions: All inputs consist of lowercase English letters.

Edge cases: Empty vector input `[]`; vector containing empty strings `[""]`; single-character strings.

2. Design the Solution

Since anagrams contain the exact same letter frequencies, sorting the characters of any word yields a canonical string key (e.g., "eat", "tea", and "ate" all become "aet"). We use an `unordered_map`

where the key is this sorted representation and the value is a list of original strings matching that key. A helper function `printGroups` iterates through the 2D vector to dynamically render the results for each test case.

3. Pseudocode

```
function printGroups(groups):
    for each group in groups:
        print group elements dynamically

function groupAnagrams(strs):
    create unordered_map map from string to list of strings
    for each str in strs:
        key = sort_characters(str)
        append str to map[key]

    result = empty list of string lists
    for each entry in map:
        append entry.value to result
    return result
```

4. Correctness

Sorting a string rearranges its characters alphabetically, mapping all words containing the same letters with the same frequencies to an identical string key. The hash map ensures that any word with an identical sorted key resolves to the exact same list bucket. Iterating sequentially through the input guarantees that elements are appended in their original relative appearance order.

5. Complexity

Time: $O(N \cdot K \log K)$, where N is the number of strings and K is the maximum length of a string, due to sorting each string of length K .

Space: $O(N \cdot K)$ to store the grouped strings within the hash map structure.

6. Test Cases

- Given example: `["eat", "tea", "tan", "ate", "nat", "bat"]` $\rightarrow$ `[["bat"], ["tan", "nat"], ["eat", "tea", "ate"]]` (*group order may vary since `unordered_map` is unordered*)
- Edge case (empty string): `[""]` $\rightarrow$ `[[""]]`
- Edge case (empty vector): `[]` $\rightarrow$ `[]`

7. Execution Screenshot

```
0. Exit
Enter your choice: 4
Input: ["eat", "tea", "tan", "ate", "nat", "bat"]
Output: [
  ["bat"]
  ["tan", "nat"]
  ["eat", "tea", "ate"]
]

Input: [""]
Output: [
  [""]
]

Input: []
Output: [
]
```

Problem 5: Meeting Rooms

1. Understand the Problem

Goal: Determine whether a person can attend all given meeting intervals without any overlap.

Input: A vector of meeting intervals `[[start, end]]`.

Output: `true` if all meetings can be attended without overlapping, `false` otherwise.

Assumptions: Meetings that begin exactly when another ends (e.g., `[1, 5]` and `[5, 10]`) are allowed and do not count as an overlap.

Edge cases: Empty intervals list `[]`; single meeting `[[1, 5]]`; meetings with zero duration `[[2, 2]]`.

2. Design the Solution

To check for overlaps, we first sort the meeting intervals by their start times in ascending order. Once sorted, any overlap must occur between adjacent meetings in the sorted array. We iterate through the sorted intervals from index 1 to $N - 1$ and check if the current meeting's start time is strictly less than the previous meeting's end time (`intervals[i][0] < intervals[i-1][1]`). If so, a conflict exists and we return `false`.

3. Pseudocode

```
function canAttendMeetings(intervals):
    if intervals is empty:
        return true

    sort intervals by start time ascending

    for i from 1 to length(intervals) - 1:
        if intervals[i].start < intervals[i-1].end:
            return false
```

```
return true
```

4. Correctness

Sorting by start time ensures that for any $i < j$, meeting i starts at or before meeting j . If meeting $i - 1$ and meeting i do not overlap, then meeting $i - 1$ ends before meeting i starts. Since all subsequent meetings $j > i$ start at or after meeting i , none of them can overlap meeting $i - 1$ either. Thus, checking only adjacent pairs is sufficient to guarantee there are no hidden overlaps across non-adjacent meetings.

5. Complexity

Time: $O(N \log N)$ because sorting the N intervals dominates the linear $O(N)$ scanning pass.

Space: $O(1)$ when using an in-place sort algorithm without allocating extra arrays.

6. Test Cases

- Overlapping meetings: $[[0, 30], [5, 10], [15, 20]] \rightarrow \text{false}$
- Non-overlapping meetings: $[[7, 10], [2, 4]] \rightarrow \text{true}$
- Consecutive meetings (touching boundary): $[[1, 5], [5, 10]] \rightarrow \text{true}$

7. Execution Screenshot

```
Enter your choice: 5
Input: [[0,30], [5,10], [15,20]] -> false
Input: [[7,10], [2,4]] -> true
Input: [[1,5], [5,10]] -> true
```

Problem 6: K Closest Points to Origin

1. Understand the Problem

Goal: Given an array of 2D coordinates (x, y) and an integer k , return the k points closest to the origin $(0, 0)$.

Distance Formula: Distance from origin is Euclidean distance $\sqrt{x^2 + y^2}$. To avoid floating-point inaccuracies, we compare squared distances $x^2 + y^2$.

Input: Vector of points $[[x, y]]$ and integer k .

Output: k closest points in any order.

Assumptions: Input points are valid integer coordinates.

Edge cases: k exceeds total number of points ($k > N$); k equals total points ($k = N$); multiple points sharing identical distances; negative coordinates.

2. Design & Approach Comparison

We compare two approaches to solve this problem:

1. **Sorting Solution:** Compute distances for all N points, sort the array based on distance in ascending order, and return the subsegment up to index k .

2. **Max-Heap Solution:** Maintain a Max-Heap of size k while iterating through all points. For each point, if the heap size exceeds k , pop the maximum element (furthest point). At the end, the heap contains the k smallest distances overall.

3. Pseudocode (Heap-Based Solution)

```
function kClosestHeap(points, k):
    if k > length(points):
        k = length(points)

    create maxHeap storing pair (squaredDistance, pointIndex)

    for i from 0 to length(points) - 1:
        dist = points[i].x^2 + points[i].y^2
        push (dist, i) onto maxHeap
        if size of maxHeap > k:
            pop top element from maxHeap

    result = list of points
    while maxHeap is not empty:
        append points[maxHeap.top().index] to result
        pop top element from maxHeap

    return result
```

4. Correctness

Euclidean distance is strictly monotonic with respect to squared distance $x^2 + y^2$, preserving relative ordering without square root computations. Clamping $k = \min(k, N)$ prevents out-of-bounds iterator access when k exceeds the total number of points. The max-heap retains at most k elements; whenever a new element is added, the element with the largest squared distance is evicted. Upon processing all N points, the remaining k points are guaranteed to have the k smallest distances overall.

5. Complexity Comparison

Metric	Full Sorting	Max-Heap (Optimal for $k \ll N$)
Time Complexity	$O(N \log N)$	$O(N \log k)$
Space Complexity	$O(N)$	$O(k)$

The Max-Heap approach is strictly superior when $k \ll N$, as it reduces both time complexity from $O(N \log N)$ to $O(N \log k)$ and extra space from $O(N)$ (due to copying input points) to $O(k)$.

6. Test Cases

- Standard example: `points = [[1,3], [-2,2]]`, `k = 1` → `[[[-2, 2]]]`
- Multi-point example: `points = [[3,3], [5,-1], [-2,4]]`, `k = 2` → `[[[3, 3], [-2, 4]]]`
(order of output points may vary)
- Edge case ($k > N$): `points = [[1,1], [2,2]]`, `k = 5` → `[[[1, 1], [2, 2]]]`

7. Execution Screenshot

```
Enter your choice: 6
Test Case 1:
Input: points = [[1,3], [-2,2]], k = 1
Output (Heap): [[-2,2]]
Output (Sort): [[-2,2]]

Test Case 2:
Input: points = [[3,3], [5,-1], [-2,4]], k = 2
Output (Heap): [[-2,4], [3,3]]
Output (Sort): [[3,3], [-2,4]]

Test Case 3 (k > N edge case):
Input: points = [[1,1], [2,2]], k = 5
Output (Heap): [[2,2], [1,1]]
Output (Sort): [[1,1], [2,2]]
```

Problem 7: K Most Frequent Values

1. Understand the Problem

Goal: Given an integer array and an integer k , return the k most frequent values. If frequencies tie, return the smaller value first.

Input: Array of integers `nums` and integer k .

Output: Array containing the top k most frequent values ordered by frequency (descending) and value (ascending on ties).

Edge Cases: k larger than the number of unique elements ($k > U$); empty input array; all elements having identical frequencies.

2. Design & Approach

1. **Frequency Mapping:** Iterate through `nums` and use a hash map (`unordered_map`) to count the frequency of each unique element in $O(N)$ time.
2. **Max-Heap Selection with Custom Comparator:** Insert all unique element-frequency pairs into a Max-Heap (`priority_queue`). We construct a custom comparator `CompareP7`:
 - Primary condition: Higher frequency elements take precedence (`a.second < b.second`).
 - Tie-breaker: If frequencies match, smaller values take precedence (`a.first > b.first`).
3. **Result Extraction:** Pop elements from the Max-Heap k times (or until the heap is empty) to collect the final output array.

3. Pseudocode

```
function topKFrequent(nums, k):
    freqMap = empty hash map
    for num in nums:
        freqMap[num] = freqMap[num] + 1
```

```

k = min(k, size of freqMap)

// Custom comparator prioritizing higher frequency, then smaller value
define Comparator:
    if a.freq != b.freq:
        return a.freq < b.freq
    return a.val > b.val

maxHeap = priority queue with Comparator
for (val, freq) in freqMap:
    push (val, freq) to maxHeap

result = []
while maxHeap is not empty and length(result) < k:
    result.append(maxHeap.top().val)
    pop maxHeap

return result

```

4. Correctness

The algorithm guarantees correctness in two stages:

- **Frequency Tracking:** The hash map accurately aggregates counts for each unique value across the input array.
- **Tie-Breaker Invariant:** In C++ `std::priority_queue`, returning `true` in the custom comparator places element `a` below element `b`. By returning `a.second < b.second`, higher frequencies naturally rise to the top. When `a.second == b.second`, returning `a.first > b.first` causes smaller numerical values to be popped first, satisfying the exact tie-breaking requirement.

5. Complexity Analysis

Phase	Time Complexity	Space Complexity
Frequency Counting	$O(N)$	$O(U)$
Heap Construction & Extraction	$O(U \log U)$	$O(U)$
Total Complexity	$O(N + U \log U)$	$O(U)$

Note: N represents total elements in `nums`, and U represents total unique elements ($U \leq N$).

6. Test Cases

- Standard & Tie-Breaker: `nums = [1, 1, 1, 2, 2, 3, 4, 4]`, `k = 2` $\rightarrow$ `[1, 2]` (*1 wins with count 3; 2 and 4 tie with count 2 $\rightarrow$ smaller value 2 wins*)
- k Exceeds Unique Elements: `nums = [5, 5, 6]`, `k = 10` $\rightarrow$ `[5, 6]`

7. Execution Screenshot

```
--- Problem 7: Top K Frequent Elements ---
Input: nums = [1, 1, 1, 2, 2, 3, 4, 4], k = 2
Output: [1, 2]

Input: nums = [5, 5, 6], k = 10 (k > unique count)
Output: [5, 6]
```

Problem 8: Merge K Sorted Integer Arrays

1. Understand the Problem

Goal: Given k sorted integer arrays, merge them into a single sorted array without repeatedly sorting the complete output array.

Input: A list of k sorted integer vectors `arrays`.

Output: A single merged vector containing all elements in non-decreasing order.

Edge Cases: Empty arrays within the list; arrays of varying lengths; $k = 0$; all arrays having single elements.

2. Design & Approach

Instead of combining all arrays and performing a full $O(N \log N)$ sort, we utilize a **Min-Heap** (priority_queue) of size at most k :

1. **Initialization:** Insert a custom struct `Element` containing `{val, arrayIdx, elementIdx}` for the first element of each non-empty array into the Min-Heap.
2. **Extraction & Replenishment:** Pop the smallest element from the heap and append its value to the result array. If the array from which this element originated has a subsequent element, push that next element into the Min-Heap.
3. **Termination:** Repeat until the Min-Heap is completely empty.

3. Pseudocode

```
struct Element:
    val: integer
    arrayIdx: integer
    elementIdx: integer

function mergeKSortedArrays(arrays):
    minHeap = priority queue ordered by smallest val

    // Initialize heap with the front of each non-empty array
    for i from 0 to length(arrays) - 1:
        if arrays[i] is not empty:
            push Element(arrays[i][0], i, 0) to minHeap

    mergedResult = []
```

```

while minHeap is not empty:
    curr = pop top from minHeap
    mergedResult.append(curr.val)

    // Push next element from the same array if it exists
    if curr.elementIdx + 1 < length(arrays[curr.arrayIdx]):
        nextVal = arrays[curr.arrayIdx][curr.elementIdx + 1]
        push Element(nextVal, curr.arrayIdx, curr.elementIdx + 1) to minHeap

return mergedResult

```

4. Correctness

The invariant maintaining global sorted order relies on the properties of sorted arrays and the Min-Heap:

- **Front-of-Array Minimum Guarantee:** Because each input array is sorted in ascending order, the current unmerged front of any array is strictly smaller than or equal to all elements deeper inside that array.
- **Global Minimum Availability:** The globally smallest remaining number across all arrays must be at the front of at least one array. Since the Min-Heap always stores the current unmerged front of every active array, the global minimum is guaranteed to be present in the heap and popped at the root ($O(1)$ lookup).

5. Complexity Analysis

Metric	Full Output Resorting	Min-Heap Merge (Optimal)
Time Complexity	$O(N \log N)$	$O(N \log k)$
Space Complexity	$O(N)$	$O(k)$

Note: N represents the total number of elements across all k arrays.

6. Test Cases

- Standard Example: $[[1, 4, 7], [2, 5, 8], [0, 3, 6, 9]] \rightarrow [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]$
- Varying Lengths & Empty Array: $[[1, 10], [], [2, 3, 4, 11], [5]] \rightarrow [1, 2, 3, 4, 5, 10, 11]$

7. Execution Screenshot

```
Enter your choice: 8
=== Problem 8: Merge K Sorted Integer Arrays ===
Test Case 1:
Input: [[1, 4, 7], [2, 5, 8], [0, 3, 6, 9]]
Output: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

Test Case 2 (Varying lengths & empty array edge case):
Input: [[1, 10], [], [2, 3, 4, 11], [5]]
Output: [1, 2, 3, 4, 5, 10, 11]
```

Problem 9: Kth Largest Element in an Array

1. Understand the Problem

Goal: Find the k -th largest element in an unsorted integer array. Note that it is the k -th largest element in sorted order, not the k -th distinct element.

Input: Array of integers `nums` and integer k .

Output: The k -th largest integer value.

Edge Cases: $k = 1$ (maximum element); $k = N$ (minimum element); array containing duplicate values; k exceeding array size ($k > N$).

2. Design & Approach

Instead of sorting the entire array in $O(N \log N)$ time, we use a **Min-Heap** (`priority.queue`) constrained to size k :

1. **Bounded Min-Heap Maintenance:** Iterate through the array and push each element into the Min-Heap.
2. **Size Eviction:** Whenever the size of the heap exceeds k , pop the top (smallest) element. This continuously evicts smaller values and ensures the heap only retains the k largest elements seen so far.
3. **Result Extraction:** After processing all N elements, the Min-Heap contains exactly the k largest elements of the array. Because it is a Min-Heap, the root (top) holds the smallest among these k elements, which is precisely the k -th largest element overall.

3. Pseudocode

```
function findKthLargest(nums, k):
    if nums is empty:
        return -1

    k = min(k, length(nums))

    // Min-heap maintaining the k largest elements
    minHeap = priority queue with smaller values on top

    for num in nums:
```

```

    push num to minHeap
    if size of minHeap > k:
        pop minHeap // Evict smallest element among current k+1

// Root of min-heap is the kth largest element overall
return minHeap.top()

```

4. Correctness & Trace Logic

The correctness relies on the filtering mechanism of the bounded Min-Heap:

- **Filtering Mechanism:** For `nums = [3, 2, 1, 5, 6, 4]` and $k = 2$, the algorithm keeps a min-heap capped at size 2. All numbers smaller than 5 (1, 2, 3, 4) are evicted during processing.
- **Final Heap State:** When execution completes, the two largest numbers overall (5 and 6) remain in the heap.
- **Min-Heap Root Property:** Since the top of a Min-Heap always exposes its minimum value, accessing `minHeap.top()` returns 5, which is the smaller of the top 2 elements and thus the 2nd largest element in the entire array.

5. Complexity Analysis

Approach	Time Complexity	Space Complexity
Full Sorting	$O(N \log N)$	$O(1)$ or $O(N)$
Bounded Min-Heap (Optimal)	$O(N \log k)$	$O(k)$

Note: Pushing and popping from a heap of size k takes $O(\log k)$ time, repeated for all N elements.

6. Test Cases

- Standard Example: `nums = [3, 2, 1, 5, 6, 4]`, $k = 2 \rightarrow 5$
- Array with Duplicates: `nums = [3, 2, 3, 1, 2, 4, 5, 5, 6]`, $k = 4 \rightarrow 4$

7. Execution Screenshot

```

--- Problem 9: Kth Largest Element ---
Test Case 1:
Input: nums = [3, 2, 1, 5, 6, 4], k = 2
Output: 5

Test Case 2 (Duplicates counting as separate positions):
Input: nums = [3, 2, 3, 1, 2, 4, 5, 5, 6], k = 4
Output: 4

```

Problem 10: Count Inversions

1. Understand the Problem

Goal: Count the total number of inversion pairs (i, j) in an array such that $i < j$ and $\text{values}[i] > \text{values}[j]$.

Input: An array of integers `values`.

Output: Total count of inversions as a 64-bit integer (`long long`).

Edge Cases: Already sorted array (0 inversions); strictly reverse-sorted array (maximum inversions $\frac{N(N-1)}{2}$); empty or single-element array; large inputs that overflow standard 32-bit integer counters.

2. Design & Approach

While a brute-force approach checks all pairs in $O(N^2)$ time (which can be used for local validation), we modify **Merge Sort** to count inversions in $O(N \log N)$ time:

1. **Divide & Conquer:** Recursively divide the array into left and right halves until subarrays contain single elements.
2. **Recursive Summing:** The total inversion count is the sum of inversions in the left half, inversions in the right half, and cross-inversions between the two halves during the merge step.
3. **Cross-Inversion Counting Trick:** During the merge step, when an element in the left half `arr[i]` is strictly greater than an element in the right half `arr[j]`, an inversion occurs. Because both halves are individually sorted, all remaining elements from `arr[i]` up to `arr[mid]` are also strictly greater than `arr[j]`. Thus, we add $(mid - i + 1)$ cross-inversions in $O(1)$ time.

3. Pseudocode

```
function mergeAndCount(arr, temp, left, mid, right):
    i = left, j = mid + 1, k = left
    invCount = 0

    while i <= mid and j <= right:
        if arr[i] <= arr[j]:
            temp[k++] = arr[i++]
        else:
            // arr[i] > arr[j] implies cross-inversions!
            temp[k++] = arr[j++]
            invCount += (mid - i + 1)

    while i <= mid: temp[k++] = arr[i++]
    while j <= right: temp[k++] = arr[j++]
    for idx from left to right: arr[idx] = temp[idx]

    return invCount

function mergeSortAndCount(arr, temp, left, right):
    invCount = 0
    if left < right:
```



```

mid = left + (right - left) / 2
invCount += mergeSortAndCount(arr, temp, left, mid)
invCount += mergeSortAndCount(arr, temp, mid + 1, right)
invCount += mergeAndCount(arr, temp, left, mid, right)
return invCount

```

4. Correctness

The correctness relies on the sorted invariant guaranteed by Divide-and-Conquer:

- **Sorted Half Guarantee:** Prior to merging, both left $[\text{left} \dots \text{mid}]$ and right $[\text{mid}+1 \dots \text{right}]$ subarrays are sorted in ascending order.
- **Single-Step Accumulation:** When $\text{arr}[i] > \text{arr}[j]$, every remaining element in the left half ($\text{arr}[i], \text{arr}[i+1], \dots, \text{arr}[\text{mid}]$) is strictly greater than $\text{arr}[j]$ and comes before it in the original array order ($i < j$). Adding $(\text{mid} - i + 1)$ accounts for all such inversion pairs simultaneously without double-counting.

5. Complexity Analysis

Approach	Time Complexity	Space Complexity
Brute-Force Pair Comparison	$O(N^2)$	$O(1)$
Modified Merge Sort (Optimal)	$O(N \log N)$	$O(N)$

Note: The primary solution achieves $O(N \log N)$ time, matching standard Merge Sort. A simple $O(N^2)$ brute-force checker can be used for verification on smaller inputs.

6. Test Cases

- Assignment Example: values = [2, 4, 1, 3, 5] → 3 (Inversions: (2,1), (4,1), (4,3))
- Fully Reversed Array: values = [5, 4, 3, 2, 1] → 10 (Maximum inversions case)

7. Execution Screenshot

```

Enter your choice: === Problem 10: Count Inversions ===
Test Case 1:
Input: [2, 4, 1, 3, 5]
Output: 3

Test Case 2 (Reversed array):
Input: [5, 4, 3, 2, 1]
Output: 10

```